Skyscraper Stack Lab - Claude Code Chat Transcript

Model: Claude Opus 5.5 (Claude Code)

> <ide_opened_file>The user opened the file c:\00 College work\5th SEM\SE\LAB 4\54_skyscrapper_stack\main.py in the IDE. This may or may not be related to the current task.</ide_opened_file>

> Read the README and all the code in this project. Then:

Find and fix the bug(s) that break the game. Explain what was wrong.
Add each feature listed in the README, one at a time, in the order listed.
After each one, briefly say what you changed and in which file.
Make sure the game runs without errors at the end, and tell me how to test each feature.
Don’t change anything the README doesn’t ask for.

● Bash(cd "/c/00 College work/5th SEM/SE/LAB 4/54_skyscrapper_stack" && ls -la && cat README* && wc -l *.py)
  ⎿ (No output)

● Bash(cd "/c/00 College work/5th SEM/SE/LAB 4/54_skyscrapper_stack" && ls -la game && cat main.py && for f in game/*.py; do echo "=== $f"; cat -n "$f"; done; git stat…)
  ⎿ (No output)

● The bug is in game/game_engine.py (54_skyscrapper_stack/game/game_engine.py). Fixing it first:

● Update(game\game_engine.py)
  ⎿ Added 1 line, removed 1 line

```
58          right = min(act.x + act.width, top_block.x + top_block.width)
59          overlap = right - left
60
61 -        is_successful_drop = overlap <= 0
61 +        is_successful_drop = overlap > 0
62
63          if is_successful_drop:
64              trimmed_width = max(10.0, overlap)
```

● Update(game\game_engine.py)
  ⎿ Added 1 line, removed 1 line

```
77
78      def handle_event(self, event):
79          if self.game_over:
80 -            if (event.type == pygame.KEYDOWN and event.key == p-ygame.K_r) or \
80 +            if (event.type == pygame.KEYDOWN and event.key in (+pygame.K_SPACE, pygame.K_r)) or \
81                  (event.type == pygame.MOUSEBUTTONDOWN and event.button == 1):
82                  self.reset()
83              return
```

● Task 2: Perfect Placement.

● Update(game\game_engine.py)
  ⎿ Added 5 lines

```
10          self.block_height = 28
11          self.base_width = 180
12
13 +        # Perfect placement tuning
14 +        self.perfect_tolerance = 6      # max pixel offset that
   + still counts as "perfect"
15 +        self.perfect_bonus = 2          # extra points awarded
   +for a perfect drop
16 +        self.restore_amount = 10        # width regained per pe
   +rfect drop once a streak is going
17 +
18          self.font_title = pygame.font.SysFont(None, 38)
19          self.font_hud = pygame.font.SysFont(None, 28)
20          self.font_big = pygame.font.SysFont(None, 46)
```

● **Update(game\game_engine.py)**
  ⎿ Added 2 lines

```
35      def reset(self):
36          self.score = 0
37          self.game_over = False
38 +        self.perfect_streak = 0
39 +        self.perfect_timer = 0
40
41          base_x = (self.width - self.base_width) // 2
42          base_y = self.height - 60
```

● **Update(game\game_engine.py)**
  ⎿ Added 24 lines, removed 3 lines

```
68          is_successful_drop = overlap > 0
69
70          if is_successful_drop:
71 -            trimmed_width = max(10.0, overlap)
72 -            new_block = Block(left, act.y, trimmed_width, self.
   -block_height, act.color, speed=0)
71 +            is_perfect = abs(act.x - top_block.x) <= self.perfe
   +ct_tolerance
72 +
73 +            if is_perfect:
74 +                # Snap flush onto the tower without trimming
75 +                self.perfect_streak += 1
76 +                new_x = top_block.x
77 +                new_width = top_block.width
78 +
79 +                # Two or more perfects in a row slowly restore
   +lost width
80 +                if self.perfect_streak >= 2 and new_width < sel
   +f.base_width:
81 +                    grow = min(self.restore_amount, self.base_w
   +idth - new_width)
82 +                    new_width += grow
83 +                    new_x -= grow / 2
84 +                    new_x = max(20.0, min(new_x, self.width - 2
   +0 - new_width))
85 +
86 +                new_block = Block(new_x, act.y, new_width, self
   +.block_height, act.color, speed=0)
87 +                self.score += 1 + self.perfect_bonus
88 +                self.perfect_timer = 60  # show the prompt for
   +~1 second
89 +            else:
90 +                self.perfect_streak = 0
91 +                trimmed_width = max(10.0, overlap)
92 +                new_block = Block(left, act.y, trimmed_width, s
   +elf.block_height, act.color, speed=0)
93 +                self.score += 1
94 +
95              self.stack.append(new_block)
74 -            self.score += 1
```

```
    96
    97              if new_block.y < 180:
    98                  shift_amount = self.block_height + 4
```

● Update(game\game_engine.py)
  └ Added 3 lines

```
   119          if not self.game_over:
   120              self.active_block.update(self.width)
   121
   122 +        if self.perfect_timer > 0:
   123 +            self.perfect_timer -= 1
   124 +
   125      def render(self, screen):
   126          screen.fill((24, 27, 36))
   127
```

● Now the HUD and the "PERFECT!" prompt. The score now includes bonus points, so I show the height and the score as two separate numbers:

● Update(game\game_engine.py)
  └ Added 11 lines, removed 1 line

```
   128          title_surf = self.font_title.render("Skyscraper Stack"
        , True, (245, 245, 245))
   129          screen.blit(title_surf, (self.width // 2 - title_surf.
        get_width() // 2, 16))
   130
   131 -        score_surf = self.font_hud.render(f"Height: {self.scor
       -e}", True, (255, 220, 80))
   131 +        tower_height = len(self.stack) - 1
   132 +        score_surf = self.font_hud.render(f"Height: {tower_hei
       +ght}   Score: {self.score}", True, (255, 220, 80))
   133          screen.blit(score_surf, (self.width // 2 - score_surf.
        get_width() // 2, 54))
   134
   135          for b in self.stack:
  ...
   138          if not self.game_over:
   139              self.active_block.render(screen)
   140
   141 +        if self.perfect_timer > 0 and not self.game_over:
   142 +            label = f"PERFECT! +{self.perfect_bonus}"
   143 +            if self.perfect_streak >= 2:
   144 +                label += f"  x{self.perfect_streak}"
   145 +            perfect_surf = self.font_big.render(label, True, (
       +255, 240, 120))
   146 +            perfect_surf.set_alpha(min(255, self.perfect_timer
       + * 8))
   147 +            rise = (60 - self.perfect_timer) // 3
   148 +            screen.blit(perfect_surf, (self.width // 2 - perfe
       +ct_surf.get_width() // 2, 100 - rise))
   149 +
   150          if self.game_over:
   151              overlay = pygame.Surface((self.width, self.height)
        , pygame.SRCALPHA)
   152              overlay.fill((0, 0, 0, 195))
```

● Update(game\game_engine.py)
  └ Added 1 line, removed 1 line

```
   155              over_surf = self.font_big.render("TOWER COLLAPSED!
        ", True, (240, 75, 75))
   156              screen.blit(over_surf, (self.width // 2 - over_sur
        f.get_width() // 2, self.height // 2 - 40))
   157
   158 -            final_surf = self.font_hud.render(f"Final Height:
       -{self.score}", True, (255, 255, 255))
   158 +            final_surf = self.font_hud.render(f"Final Height:
```

```
        +{tower_height}   Score: {self.score}", True, (255, 255, 255))
    159             screen.blit(final_surf, (self.width // 2 - final_s
         urf.get_width() // 2, self.height // 2 + 10))
    160
    161             restart_surf = self.font_hud.render("Press [Space]
          or [R] to Play Again", True, (200, 200, 200))
```

● **Task 3: falling debris. I'm adding a Debris class to game/block.py** (54_skyscrapper_stack/game/block.py):

● **Bash(cd "/c/00 College work/5th SEM/SE/LAB 4/54_skyscrapper_stack" && cat >>** game/block.py <<'EOF'…)

  └ Updated game\block.py (+33 -1)

```
    29      def render(self, surface):
    30          draw_rect = self.rect
    31          pygame.draw.rect(surface, self.color, draw_rect, border
         _radius=4)
    32 -        pygame.draw.rect(surface, (245, 245, 250), draw_rect, w
       -idth=2, border_radius=4)
    32 +        pygame.draw.rect(surface, (245, 245, 250), draw_rect, w
       +idth=2, border_radius=4)
    33 +
    34 +class Debris:
    35 +    """A trimmed-off block section that falls and spins out of
       +the arena."""
    36 +
    37 +    def __init__(self, x, y, width, height, color, direction):
    38 +        self.x = float(x)
    39 +        self.y = float(y)
    40 +        self.width = max(1.0, float(width))
    41 +        self.height = float(height)
    42 +        self.color = color
    43 +        self.vx = 1.5 * direction       # drift away from the t
       +ower
    44 +        self.vy = -1.5                  # small upward pop befo
       +re falling
    45 +        self.angle = 0.0
    46 +        self.spin = 4.0 * direction
    47 +        self.gravity = 0.45
    48 +
    49 +    def update(self):
    50 +        self.vy += self.gravity
    51 +        self.x += self.vx
    52 +        self.y += self.vy
    53 +        self.angle += self.spin
    54 +
    55 +    def is_off_screen(self, screen_height):
    56 +        return self.y > screen_height + 100
    57 +
    58 +    def render(self, surface):
    59 +        piece = pygame.Surface((int(self.width), int(self.heigh
       +t)), pygame.SRCALPHA)
    60 +        piece.fill(self.color)
    61 +        pygame.draw.rect(piece, (245, 245, 250), piece.get_rect
       +(), width=2)
    62 +        rotated = pygame.transform.rotate(piece, self.angle)
    63 +        center = (self.x + self.width / 2, self.y + self.height
       + / 2)
    64 +        surface.blit(rotated, rotated.get_rect(center=center))
```

● **Now I'm wiring it into the engine. Debris spawns for each overhanging side,** shifts along with the camera, and keeps falling on the collapse screen:

● **Update(game\game_engine.py)**

  └ Added 1 line, removed 1 line

```
    1  import random
```

```
2  import pygame
3 -from game.block import Block
3 +from game.block import Block, Debris
4
5
6  class GameEngine:
```

● Update(game\game_engine.py)
  └ Added 1 line

```
37          self.game_over = False
38          self.perfect_streak = 0
39          self.perfect_timer = 0
40 +        self.debris = []
41
42          base_x = (self.width - self.base_width) // 2
43          base_y = self.height - 60
```

● Update(game\game_engine.py)
  └ Added 7 lines

```
 93                  new_block = Block(left, act.y, trimmed_width,
     self.block_height, act.color, speed=0)
 94                  self.score += 1
 95
 96 +                # Spawn falling debris for whichever side(s) h
    +ung over the edge
 97 +                if act.x < left:
 98 +                    self.debris.append(Debris(act.x, act.y, le
    +ft - act.x, self.block_height, act.color, -1))
 99 +                act_right = act.x + act.width
100 +                if act_right > right:
101 +                    self.debris.append(Debris(right, act.y, ac
    +t_right - right, self.block_height, act.color, 1))
102 +
103              self.stack.append(new_block)
104
105              if new_block.y < 180:
```

● Update(game\game_engine.py)
  └ Added 2 lines

```
106                  shift_amount = self.block_height + 4
107                  for b in self.stack:
108                      b.y += shift_amount
109 +                for d in self.debris:
110 +                    d.y += shift_amount
111
112              self.spawn_active_block()
113          else:
```

● Update(game\game_engine.py) └ Added 4 lines

```
132          if self.perfect_timer > 0:
133              self.perfect_timer -= 1
134
135 +        for d in self.debris:
136 +            d.update()
137 +        self.debris = [d for d in self.debris if not d.is_off_
    +screen(self.height)]
138 +
139      def render(self, screen):
140          screen.fill((24, 27, 36))
141
```

● Update(game\game_engine.py)
  └ Added 3 lines

```
149          for b in self.stack:
150              b.render(screen)
```

```
151
152 +        for d in self.debris:
153 +            d.render(screen)
154 +
155          if not self.game_over:
156              self.active_block.render(screen)
157
```

● **Task 4: background that changes with height (in game/game_engine.py** (54_skyscrapper_stack/game/game_engine.py)).

● **Update(game\game_engine.py)**

└ Added 10 lines

```
15          self.perfect_bonus = 2          # extra points awarded
    for a perfect drop
16          self.restore_amount = 10        # width regained per pe
    rfect drop once a streak is going
17
18 +        # Atmospheric sky stages as (top color, bottom color),
   +from ground level to space
19 +        self.sky_stages = [
20 +            ((70, 130, 190), (150, 195, 230)),   # Daytime
21 +            ((220, 110, 90), (250, 180, 110)),   # Sunset
22 +            ((80, 50, 120), (190, 90, 120)),     # Twilight
23 +            ((15, 20, 55), (40, 50, 100)),       # Night
24 +            ((2, 2, 10), (15, 10, 35)),          # Upper atmosp
   +here / space
25 +        ]
26 +        self.floors_per_stage = 8
27 +
28          self.font_title = pygame.font.SysFont(None, 38)
29          self.font_hud = pygame.font.SysFont(None, 28)
30          self.font_big = pygame.font.SysFont(None, 46)
```

● **Update(game\game_engine.py)**

└ Added 1 line

```
48          self.perfect_streak = 0
49          self.perfect_timer = 0
50          self.debris = []
51 +        self.sky_progress = 0.0
52
53          base_x = (self.width - self.base_width) // 2
54          base_y = self.height - 60
```

● **Update(game\game_engine.py)**

└ Added 19 lines

```
147              d.update()
148          self.debris = [d for d in self.debris if not d.is_off_
     screen(self.height)]
149
150 +        # Ease the sky toward the stage matching the current t
    +ower height
151 +        target = min((len(self.stack) - 1) / self.floors_per_s
    +tage, len(self.sky_stages) - 1)
152 +        self.sky_progress += (target - self.sky_progress) * 0.
    +03
153 +
154 +    @staticmethod
155 +    def lerp_color(a, b, t):
156 +        return tuple(int(a[i] + (b[i] - a[i]) * t) for i in ra
    +nge(3))
157 +
158 +    def draw_sky(self, screen):
159 +        stage = min(int(self.sky_progress), len(self.sky_stage
    +s) - 2)
160 +        t = self.sky_progress - stage
```

```
161 +        top_a, bottom_a = self.sky_stages[stage]
162 +        top_b, bottom_b = self.sky_stages[stage + 1]
163 +        top = self.lerp_color(top_a, top_b, t)
164 +        bottom = self.lerp_color(bottom_a, bottom_b, t)
165 +
166 +        for y in range(self.height):
167 +            pygame.draw.line(screen, self.lerp_color(top, bott
    +om, y / self.height), (0, y), (self.width, y))
168 +
169      def render(self, screen):
170          screen.fill((24, 27, 36))
171
```

● **Update(game\game_engine.py)**

└ Added 1 line, removed 1 line

```
167              pygame.draw.line(screen, self.lerp_color(top, bott
     om, y / self.height), (0, y), (self.width, y))
168
169      def render(self, screen):
170 -        screen.fill((24, 27, 36))
170 +        self.draw_sky(screen)
171
172          title_surf = self.font_title.render("Skyscraper Stack"
     , True, (245, 245, 245))
173          screen.blit(title_surf, (self.width // 2 - title_surf.
     get_width() // 2, 16))
```

● **Now I'm running a headless test script that drives every code path:**

● **Bash(cd "/c/00 College work/5th SEM/SE/LAB 4/54_skyscrapper_stack" && cat >**
"$TEMP/claude/c--00-College-work-5th-SEM-SE-LAB-4/7c928c42-fe11-40a9-9792-aa2f7da8f149/sc…)

└ Error: Exit code 2

```
/usr/bin/bash: line 34:
C:\Users\Sushanth\AppData\Local\Temp/claude/c--00-College-work-5th-SEM-SE-LAB-4/7c928c42-fe11-40a9-9792-aa2f7da8f149/scratchpad/t.py: No such file or directory
C:\Python313\python.exe: can't open file
'C:\\Users\\Sushanth\\AppData\\Local\\Temp/claude/c--00-College-work-5th-SEM-SE-LAB-4/7c928c42-fe11-40a9-9792-aa2f7da8f149/scratchpad/t.py': [Errno 2] No such file or directory
```

● **Bash(cd "/c/00 College work/5th SEM/SE/LAB 4/54_skyscrapper_stack" &&**
S="/c/Users/Sushanth/AppData/Local/Temp/claude/c--00-College-work-5th-SEM-SE-LAB-4/7c928c42-fe1…)

└ 2 files changed (binary, mode only or too large to show)

● **I fixed the bug and added all three features. A headless test script dropped** blocks offset, perfectly aligned, and completely missing, then restarted, and every check passed with no errors. I didn't open the game window myself, so play it once to confirm it looks right.

The bugs

Main bug (Task 1): the drop check was backwards. In game_engine.py:61 (54_skyscrapper_stack/game/game_engine.py#L61) the code read `is_successful_drop = overlap <= 0`. overlap is how much of the dropped block sits over the top of the tower. So the old check counted a drop as good only when there was no overlap at all. Landing on the tower ended the game, and dropping into empty air stacked a block. I changed it to `overlap > 0`.

Smaller bug: Space didn't restart. The README and the collapse screen both say "Press [Space] or [R]", but the code only restarted on R or a left-click. I added Space to the restart keys. Left-click still restarts too, as before.

Features, in README order

Task 2: Perfect placement (file: game/game_engine.py
(54_skyscrapper_stack/game/game_engine.py))
- A drop counts as perfect if it's within 6 px of the block below. It snaps
  flush with no trimming.
- A perfect drop gives +2 bonus points and a "PERFECT! +2" message that floats
  up and fades. From the second perfect drop in a row, it also shows the
  streak (e.g. "x2").
- From the second perfect drop in a row, the block regrows 10 px wide, up to
  the starting width of 180.
- Because bonus points mean the score no longer equals the number of floors,
  the top line now reads Height: N   Score: S. The collapse screen shows both
  as well.

Task 3: Falling debris (files: game/block.py
(54_skyscrapper_stack/game/block.py) and game/game_engine.py
(54_skyscrapper_stack/game/game_engine.py))
- In block.py I added a Debris class: a piece that pops up slightly, drifts
  away from the tower, spins and falls.
- In game_engine.py, the part of the block that hung over the edge now becomes
  debris instead of disappearing. Debris moves down with the camera, keeps
  falling on the collapse screen, and is removed once it leaves the screen.

Task 4: Changing sky (file: game/game_engine.py
(54_skyscrapper_stack/game/game_engine.py))
- The plain dark background is now a top-to-bottom colour gradient. It moves
  through five stages as the tower grows, one every 8 floors: day, sunset,
  twilight, night, then near-black space.
- The colours blend gradually rather than switching suddenly.

How to test (python main.py)

What: Bug fix
How to check it: Drop a block that's partly over the tower: it stacks and the
edges are trimmed. Drop one completely off the tower: "TOWER COLLAPSED!"
appears.
———————————————————————————————

What: Restart
How to check it: On the collapse screen, press Space or R: you're back to a
single full-width base block.
———————————————————————————————

What: Perfect placement
How to check it: Drop a block almost exactly on the one below. You should see
"PERFECT! +2", no trimming, and the score going up by 3. If you get two or
more in a row after a trimmed drop, the block gets slightly wider each time.
It's easier on the first few floors, while the blocks move slowly.
———————————————————————————————

What: Debris
How to check it: Drop a block clearly off-centre. The cut-off piece falls and
spins off the screen on the side it hung over. A perfect drop creates no
debris.
———————————————————————————————

What: Sky
How to check it: Stack about 8 floors: the sky turns to sunset. Around 16
floors it's twilight, around 24 night, and around 32 space.

Nothing else was changed. The camera scrolling, the speed increase and the
block colours are as they were.